

US35 - Worst-Case Time Complexity Analysis

sem2-pi-25-26-g023 · Sprint 3

Algorithms: PathFinder (US34) · SubnetworkExtractor (US36)

1. Introduction

This document analyses the worst-case time complexity of the two algorithms developed in Sprint 3 for the Transparency Portal network graph. Both operate on the support graph, an undirected unweighted graph $G = (V, E)$ derived from the directed relations graph by treating every directed edge as bidirectional and discarding weights. The support graph is stored as a boolean adjacency matrix of dimension $N \times N$, where $N = |V|$.

The two algorithms are:

- `PathFinder.shortestDistance` (US34) – determines whether a pathway exists between two entities and returns the shortest distance.
- `SubnetworkExtractor.extract` (US36) – identifies the connected component that contains a given entity, representing all nodes that can participate in influence chains passing through it.

Throughout this document N denotes the number of nodes and E the number of undirected edges in the support graph. Because the graph is stored as an adjacency matrix, iterating the neighbours of any node requires scanning an entire row of N entries, regardless of the node's actual degree.

2. Support Graph Construction (SupportGraph)

The support graph is built once from the directed `RelationGraph` before either algorithm executes. This construction step is shared by US34 and US36.

2.1 Pseudocode

```
BuildSupportGraph(RelationGraph RG):
    // Pass 1 - register all N node ids
    for each node u in RG.nodes():           // N iterations
        registry.indexFor(u)                 // 0(N) scan of ids list

    allocate adj[N][N] = false               // 0(N^2)

    // Pass 2 - fill adjacency matrix
    for each node u in RG.nodes():           // N iterations
        i = registry.indexFor(u)             // 0(N)
        for each edge e in RG.neighbors(u):  // deg(u) iterations
            j = registry.indexFor(e.toId)     // 0(N)
            if i != j:
                adj[i][j] = true
                adj[j][i] = true              // symmetric

    return SupportGraph(adj, registry)
```

2.2 Complexity Analysis

Step	Cost	Reason
Pass 1: index registration	$O(N^2)$	N nodes, each requiring an $O(N)$ linear scan of the id list
Allocate $N \times N$ matrix	$O(N^2)$	Initialising the <code>boolean[N][N]</code> array
Pass 2: fill adjacency matrix	$O(N^2)$	N nodes; per node $O(N)$ for <code>indexOf</code> $\times$ <code>deg(u)</code> edges; sum over all nodes
Total	$O(N^2)$	Dominated by both passes

If `IndexRegistry` used a hash map instead of a linear list, each `indexOf` call would cost $O(1)$ and total construction would reduce to $O(N + E)$. The linear-list implementation was chosen to keep all data structures primitive, consistent with the AC requirements of US34 and US36.

3. US34 - `PathFinder.shortestDistance`

This method determines whether a pathway exists between two entities in the support graph and, if so, returns the length of the shortest path in edges. It is implemented as a breadth-first search (BFS) using only primitive Java constructs to comply with AC3: no `Queue`, `LinkedList`, `ArrayDeque` or any other `java.util` collection class is used. The BFS queue is a plain `int[]` array managed with manual head and tail pointers.

3.1 Pseudocode

```
PathFind(G, src, dst):
    if src == dst: return 0                // 0(1)

    dist[0..N-1] = -1                      // 0(N)  initialise
    queue[0..N-1] = unset int array       // 0(N)  allocate
    head = 0; tail = 0

    dist[src] = 0
    queue[tail] = src; tail = tail + 1     // enqueue src

    while head != tail:                    // at most N dequeues
        u = queue[head]; head = head + 1

        for v = 0 to N-1:                 // full row scan 0(N)
            if adj[u][v] AND dist[v] == -1:
                dist[v] = dist[u] + 1
                if v == dst: return dist[v] // early exit
                queue[tail] = v; tail = tail + 1

    return -1                             // no path
```

3.2 Complexity Analysis

Step	Cost	Reason
Initialise <code>dist[]</code> and <code>queue[]</code>	$O(N)$	One pass over N elements for each array
BFS outer loop (dequeue)	$O(N)$	Each of the N nodes is enqueued and dequeued at most once
BFS inner loop (row scan)	$O(N)$ per dequeue	Full adjacency-matrix row scanned for every dequeued node
Total BFS work	$O(N^2)$	N dequeues $\times$ $O(N)$ inner scan

Total PathFind	$O(N^2)$	Dominated by the BFS on the adjacency matrix
----------------	----------	--

3.3 Worst-Case Scenario

The worst case arises when no path exists between source and target, forcing BFS to exhaust all N nodes before concluding. Every node is dequeued once and its full adjacency row (N entries) is scanned, yielding exactly N^2 matrix accesses in the worst case.

Had the graph been stored as an adjacency list, the inner loop would cost $O(\deg(u))$ per dequeue and the total complexity would reduce to $O(N + E)$. The matrix representation was retained to satisfy the primitive-operations constraint of AC3 and is appropriate for the network sizes encountered in this application.

4. US36 - SubnetworkExtractor.extract

This method extracts the connected component of the support graph that contains a given origin entity. All nodes in this component can participate in influence chains passing through the origin, making the component the natural boundary for the subnetwork to be visualised. The algorithm uses only primitive arrays to comply with AC2.

4.1 Pseudocode

```

ExtractSubnetwork(G, origin):
    // Phase 1 - BFS to mark all reachable nodes
    visited[0..N-1] = false           // 0(N)
    queue[0..N-1] = unset int array  // 0(N)
    head = 0; tail = 0

    src = registry.indexFor(origin)    // 0(N)
    visited[src] = true
    queue[tail] = src; tail = tail + 1

    while head != tail:                // at most N dequeues
        u = queue[head]; head = head + 1
        for v = 0 to N-1:              // 0(N) per dequeue
            if adj[u][v] AND NOT visited[v]:
                visited[v] = true
                queue[tail] = v; tail = tail + 1

    // Phase 2 - collect visited node ids
    count = 0
    for i = 0 to N-1:                  // 0(N)
        if visited[i]: count = count + 1

    subIndex[0..N-1] = -1              // 0(N)
    subIds[0..count-1] = empty
    k = 0
    for i = 0 to N-1:                  // 0(N)
        if visited[i]:
            subIndex[i] = k
            subIds[k] = registry.idAt(i) // 0(1)

```

```

    k = k + 1

    // Phase 3 - build sub-adjacency matrix
    subAdj[count][count] = false           // 0(count^2) <= 0(N^2)
    for i = 0 to N-1:                       // 0(N^2)
        if NOT visited[i]: continue
        for j = 0 to N-1:
            if NOT visited[j]: continue
            subAdj[subIndex[i]][subIndex[j]] = adj[i][j]

    return SubnetworkResult(origin, subIds, subAdj)

```

4.2 Complexity Analysis

Step	Cost	Reason
Phase 1 - BFS	$O(N^2)$	Same analysis as PathFinder: N dequeues $\times$ $O(N)$ row scan
Phase 2 - collect visited nodes	$O(N)$	Three sequential linear passes over the N -element arrays
Phase 3 - build sub-adjacency	$O(N^2)$	Double loop over all N indices; skips unvisited nodes but still
Total ExtractSubnetwork	$O(N^2)$	Phase 1 and Phase 3 both contribute $O(N^2)$; Phase 2 is dominated

4.3 Worst-Case Scenario

The worst case occurs when the graph is fully connected. BFS then visits all N nodes (Phase 1: $O(N^2)$), and Phase 3 copies the complete $N \times N$ matrix (also $O(N^2)$). The resulting subnetwork is identical to the full graph and $\text{count} = N$.

When the origin is completely isolated, Phases 2 and 3 degenerate to $O(N)$ and $O(1)$ respectively. Phase 1, however, still traverses the full adjacency matrix before terminating, so the overall cost remains $O(N^2)$.

5. Summary and Comparison

Algorithm	Class	Time	Space
BuildSupportGraph	SupportGraph	$O(N^2)$	$O(N^2)$
PathFind (US34)	PathFinder	$O(N^2)$	$O(N)$
ExtractSubnetwork (US36)	SubnetworkExtractor	$O(N^2)$	$O(N^2)$

All three algorithms are $O(N^2)$ in the worst case. This follows directly from the adjacency-matrix representation: scanning the neighbours of any node requires a full $O(N)$ row traversal, so a BFS that visits all N nodes performs N^2 matrix accesses overall.

PathFinder requires only $O(N)$ auxiliary space (the `dist[]` and `queue[]` arrays). Both SupportGraph and SubnetworkExtractor allocate an $N \times N$ boolean matrix, giving $O(N^2)$ space.

5.1 Tightness of the Bound

The $O(N^2)$ bound is tight for a complete graph K_n , where every node has $N-1$ neighbours and $E = N(N-1)/2$. In this configuration BFS dequeues all N nodes and the inner loop finds $N-1$ true entries per row, giving exactly N^2 adjacency-matrix accesses. Phase 3 of `ExtractSubnetwork` similarly copies N^2 entries.

For sparse graphs where $E \ll N^2$, the algorithms still cost $O(N^2)$ because the row scan always traverses all N columns regardless of how many true entries it finds. An adjacency-list implementation would reduce both BFS algorithms to $O(N + E)$ in that case.

6. Acceptance Criteria Compliance

US34 AC3 and US36 AC2 require all procedures to use primitive operations only, without Java library functions such as boolean or matrix multiplication from existing libraries. The following points confirm compliance.

- No `java.util.Queue`, `LinkedList`, `ArrayDeque`, or `Collections` class is used in `PathFinder` or `SubnetworkExtractor`.
- The BFS queue is implemented as a plain `int[]` array with manual head and tail index variables.
- Visited and distance state is tracked with `boolean[]` and `int[]` arrays.
- Adjacency-matrix row iteration uses a plain for loop over integer indices.
- The sub-adjacency matrix in `SubnetworkExtractor` is built with nested for loops; no `Arrays.copyOf` or similar utility is called.
- Defensive copies in `SubnetworkResult` are made with primitive array loops.

Regarding the visualisation step: `SubnetworkController.exportToSvg()` delegates to `GraphDotExporter` and invokes `Graphviz` via a system process. This is explicitly permitted by US36 AC2, which excludes the procedure used for graphic visualisation from the primitive-operations constraint.

End of document